

Тема урока: композиция

1. Композиция
2. Композиция и наследование

Аннотация. Урок посвящен композиции при проектировании классов.

Композиция

В объектно-ориентированных языках программирования существует несколько способов организации взаимодействия между классами, одним из которых является наследование. Суть наследования заключается в создании дочернего класса, имеющего все атрибуты и методы родительского класса. Наследование описывается словом **является**: кошка является животным, автомобиль является транспортом.

В этом уроке мы поговорим еще об одном способе — **композиции**. Композиция представляет собой ситуацию, когда экземпляр одного класса **включает в себя** экземпляр другого класса. Класс, экземпляр которого содержит экземпляр другого класса, называют **составным**. Класс, экземпляр которого содержится в экземпляре другого класса, называют **компонентным**. Композиция описывается словом **имеет**: кошка имеет лапы, автомобиль имеет двигатель.



Композиция позволяет составному классу использовать функционал компонентного класса, не наследуясь от него. Примером может служить класс `UserDict` из модуля `collections`, экземпляр которого внутренне содержит экземпляр класса `dict`, используя функционал которого реализует свой.

В отличие от наследования, композиция не выстраивает строгие иерархические отношения между классами. Это означает, что изменения в компонентном классе редко влияют на составной класс, а изменения в составном классе никогда не влияют на компонентный класс. При рассмотрении программ, одна из которых основана на наследовании, а другая — на композиции, композиционное решение обычно является наиболее гибким.

Рассмотрим несколько примеров композиции и начнем с самого простого — класса `Rectangle`, описывающего прямоугольник.

Приведенный ниже код:

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length           # длина
прямоугольника
        self.width = width             # ширина
прямоугольника

rectangle = Rectangle(1, 2)

print(type(rectangle))
print(type(rectangle.length))
print(type(rectangle.width))
```

ВЫВОДИТ:

```
<class '__main__.Rectangle'>
<class 'int'>
<class 'int'>
```

Экземпляры класса `Rectangle` содержат информацию о сторонах прямоугольника, которые в нашем случае представляются экземплярами класса `int`. Таким образом, классы `Rectangle` и `int` образуют собой композицию, где класс `Rectangle` является составным, а `int` — компонентным.

Мы можем определить в классе `Rectangle` набор методов для вычисления периметра и площади прямоугольника, используя имеющиеся данные о его сторонах.

Приведенный ниже код:

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def perimeter(self):
        return (self.length + self.width) * 2           # периметр
прямоугольника

    def area(self):
        return self.length * self.width                 # площадь
прямоугольника

rectangle = Rectangle(1, 2)

print(rectangle.perimeter())
print(rectangle.area())
```

ВЫВОДИТ:

Далее перейдем к случаю, когда компонентный класс является пользовательским, а не встроенным. Для этого рассмотрим класс `Square`, описывающий квадрат, который внутренне содержит экземпляр класса `Rectangle` с равными значениями длины и ширины.

Приведенный ниже код:

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def perimeter(self):
        return (self.length + self.width) * 2

    def area(self):
        return self.length * self.width

class Square:
    def __init__(self, side):
        self.value = Rectangle(side, side)           # прямоугольник с
        равными сторонами

square = Square(2)

print(type(square))
```

ВЫВОДИТ:

```
<class '__main__.Square'>
```

Класс `Square` не является наследником класса `Rectangle`, поэтому экземпляр класса `Square` не имеет методов `perimeter()` и `area()` класса `Rectangle`, однако он внутренне содержит экземпляр класса `Rectangle`, через который может пользоваться этими методами. Таким образом, класс `Square` может определить собственные методы `perimeter()` и `area()`, но не реализовывать их функционал.

Приведенный ниже код:

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def perimeter(self):
        return (self.length + self.width) * 2

    def area(self):
        return self.length * self.width

class Square:
    def __init__(self, side):
        self.value = Rectangle(side, side)

    def perimeter(self):
        return self.value.perimeter()

    def area(self):
        return self.value.area()

square = Square(2)

print(square.perimeter())
print(square.area())
```

ВЫВОДИТ:

```
8
4
```



Композиция — это слабосвязанные отношения, которые часто не требуют, чтобы составной класс знал о реализации компонентного класса.

Изобразить взаимосвязь между классами `Rectangle` и `Square`, а также классом `int`, являющимся компонентным для первого, можно следующим образом:

Rectangle

`length: int`
`width: int`

Square

`value: Rectangle`

В начале урока мы упомянули, что композиция описывается словом **имеет**: кошка имеет лапы, автомобиль имеет двигатель. Однако пример выше недостаточно наглядно демонстрирует это отношение, так как трудно в неформальном виде представить квадрат, владеющий прямоугольником. Поэтому в качестве второго примера, сперва без композиции, рассмотрим класс `Employee`, описывающий наемного работника:

```
class Employee:
    def __init__(self, name, pay, bonus=0):
        self.name = name           # имя работника
        self.pay = pay             # ежемесячная
        зарплата
        self.bonus = bonus         # ежегодная премия,
        по умолчанию равна 0
```

Определим в этом классе метод `total_salary()`, возвращающий сумму, которую работник получает за год.

Приведенный ниже код:

```
class Employee:
    def __init__(self, name, pay, bonus=0):
        self.name = name
        self.pay = pay
        self.bonus = bonus

    def total_salary(self):
        return 12 * self.pay + self.bonus           # годовая зарплата

employee = Employee('Гвидо', 100000, 10000)

print(employee.total_salary())
```

Выводит:

```
1210000
```

Несмотря на то что класс `Employee` работает как и требуется, он выполняет неприсущие ему операции, а именно вычисляет годовую заработную плату, хотя его задачей по большей части является лишь хранение этого значения. Поэтому мы можем определить класс `Salary`, описывающий зарплату, который содержит набор методов, выполняющих необходимые вычисления, а в классе `Employee` лишь пользоваться этими методами.

Приведенный ниже код:

```

class Salary:
    def __init__(self, pay, bonus):
        self.pay = pay                # ежемесячная
зарплата
        self.bonus = bonus            # ежегодная премия

    def annual_salary(self):
        return 12 * self.pay + self.bonus    # годовая зарплата

class Employee:
    def __init__(self, name, pay, bonus):
        self.name = name
        self.salary = Salary(pay, bonus)      # объект, содержащий
все данные о зарплате

    def total_salary(self):
        return self.salary.annual_salary()

employee = Employee('Гвидо', 100000, 10000)

print(employee.total_salary())

```

ВЫВОДИТ:

```
1210000
```

Здесь взаимосвязь между классами, которая описывается словом **имеет**, более наглядна, так как работник, представленный экземпляром класса `Employee`, буквально имеет имя и имеет заработную плату, представленные экземплярами классов `str` и `Salary` соответственно.



Композиция — это построение целого из частей с минимальной взаимосвязью.

Может возникнуть вопрос о целесообразности применения композиции в данном случае, ведь мы определили дополнительный класс и, по сути, увеличили количество кода вдвое. Однако стоит отметить, что сам код стал проще, так как вычисления в классе `Employee` были заменены на вызов говорящего метода. Также, если в дальнейшем нам нужно будет производить какие-либо операции с зарплатой, например, вычислять полугодовое значение без премии, мы снова определим соответствующий метод в классе `Salary` и лишь воспользуемся им в классе `Employee`.

Композиция и наследование

Частым вопросом является выбор между композицией и наследованием, ведь они оба позволяют повторно использовать код и могут решать аналогичные проблемы. Зачастую в качестве правила, определяющего отношение между классами, предлагается выбор слова, связывающего эти классы: **является** или **имеет**. Если подходит первое слово, то предпочтение стоит отдать наследованию, если второе — композиции.

Наследование используется, когда класс хочет получить весь функционал родительского класса а затем расширить его, определив новые методы, или изменить, переопределив уже имеющиеся. В случае с композицией мы можем только использовать этот класс и не имеем возможности его изменить. Таким образом, когда нужно использовать класс как таковой без каких-либо модификаций, рекомендуется композиция, а когда нужно изменить поведение метода в другом классе, рекомендуется наследование.

Общий совет заключается в использовании отношения, которое создает меньше зависимостей между двумя классами. Этим отношением является композиция. Не путайте композицию с наследованием. Композиция означает, что объект знает другой объект и явно делегирует ему некоторые задачи.

Примечания

Примечание 1. Нередко в программировании ситуацию, когда экземпляр одного класса включает в себя экземпляр другого класса, разделяют на два типа: композицию и агрегацию. В первом случае считается, что экземпляр компонентного класса создается внутри экземпляра составного класса и не существует отдельно от него. Во втором случае, наоборот, экземпляр компонентного класса создается отдельно от экземпляра составного класса и передается в качестве аргумента при создании второго.

Пример композиции:

```
class Salary:
    def __init__(self, pay, bonus):
        self.pay = pay
        self.bonus = bonus

class Employee:
    def __init__(self, name, pay, bonus):
        self.name = name
        self.salary = Salary(pay, bonus)

employee = Employee('Гвидо', 100000, 10000)
```

Пример агрегации:

```
class Salary:
    def __init__(self, pay, bonus):
        self.pay = pay
        self.bonus = bonus

class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
```

```
salary = Salary(100000, 10000)
employee = Employee('Faisal', salary)
```


